USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Memory Map and Allocation Policy

UM-NATOS-004

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-004	1.0 · 2026-08-14	current — **contains one open risk (§5)**	Internal

1. Abstract

This report records the ESP32 address regions relevant to nat-os, the layout chosen in `kernel/linker.ld`, the reasoning behind each placement, and one unresolved overlap between the kernel's IRAM region and the bootloader's own working memory.

2. ESP32 internal memory regions

The ESP32 has 520 KB of internal SRAM in three blocks, some of which is visible at more than one address depending on whether it is accessed as instruction or data memory.

Block	Size	Data (DRAM) view	Instruction (IRAM) view
SRAM0	192 KB	—	`0x40070000` – `0x400A0000`
SRAM1	128 KB	`0x3FFE0000` – `0x40000000`	`0x400A0000` – `0x400C0000`
SRAM2	200 KB	`0x3FFAE000` – `0x3FFE0000`	—

External flash is additionally mapped through the cache:

Purpose	Address base
Flash instruction (execute in place)	`0x400D0000`
Flash read-only data	`0x3F400000`

> **Verification note.** These boundaries are transcribed from the ESP32 Technical Reference Manual's system address mapping and should be checked against it before being relied on for anything beyond the current layout. The addresses nat-os actually uses (§3) are confirmed working on hardware; the surrounding region boundaries are not independently verified.

2.1 Two properties that constrain layout

IRAM requires aligned 32-bit access. Byte or unaligned halfword loads from IRAM fault. Read-only data — notably string literals — must therefore live in DRAM, not IRAM, even though it is logically part of the program image.

Part of SRAM0 serves as instruction cache. When flash execute-in-place is enabled, a portion of SRAM0 is consumed by the cache and is unavailable as IRAM. M0 does not execute from flash, so this does not currently apply, but it reduces available IRAM once it does.

3. nat-os layout

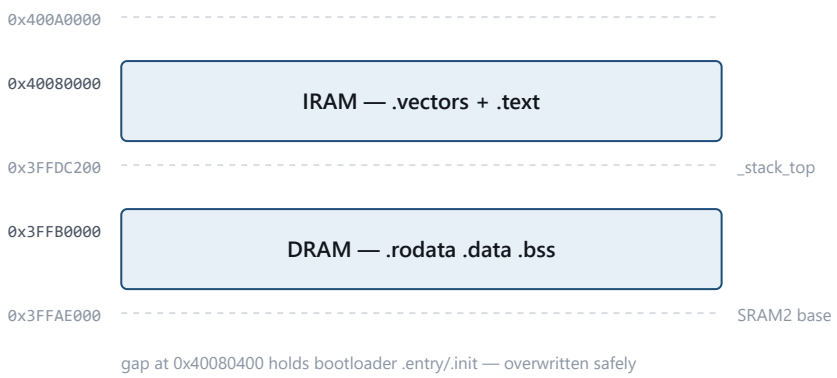

Figure 1. *Kernel address layout. .rodata is kept out of IRAM because IRAM cannot serve unaligned reads.*

Declared in `kernel/linker.ld`:

```
MEMORY
{
  iram (rwx) : ORIGIN = 0x40080000, LENGTH = 0x20000   /* 128 KB */
  dram (rw)  : ORIGIN = 0x3FFB0000, LENGTH = 0x2C200   /* ~176 KB */
}
```

Section	Region	Rationale
`.text` (code + literals)	IRAM	Executable; M0 runs entirely from RAM, no cache dependency
`.rodata`	DRAM	**Required** — IRAM cannot serve unaligned reads (§2.1)
`.data`	DRAM	Writable initialised data
`.bss`	DRAM	Zeroed by `start.S`; `NOLOAD`, so it costs no image bytes
Stack	DRAM, top-down from `_stack_top`	Placed at `ORIGIN + LENGTH` = `0x3FFDC200`

3.1 Measured occupancy (M0)

Section	Size	Region
`.text`	1,124 B	IRAM
`.data`	4 B	DRAM
`.bss`	4 B	DRAM
Image total	1,216 B	flash @ `0x10000`

Confirmed on hardware: `.bss` spans `0x3FFB0188` – `0x3FFB018C`, stack top `0x3FFDC200`, and executing code was observed at `0x40080088` — inside the declared IRAM window.

3.2 Why DRAM starts at `0x3FFB0000` and not `0x3FFAE000`

SRAM2 begins at `0x3FFAE000`, but the origin is set 8 KB higher. The upper region of DRAM (above roughly `0x3FFE0000`) is used by the ROM for its own stack and data during boot; starting low and ending below that region avoids being overwritten while the bootloader is still running. The chosen length (`0x2C200`) places the top at `0x3FFDC200`, comfortably clear.

4. Allocation policy (forward-looking)

No allocator exists yet. The intended policy, to be implemented at M3:

Consumer	Region	Notes
Kernel code, hot paths	IRAM	Interrupt handlers and scheduler must not depend on flash cache
Kernel code, cold paths	Flash XIP	Once cache configuration is owned
Kernel data and stacks	DRAM low	One stack per native task
VM application arenas	DRAM, fixed-size blocks	**Bounds-checked by the interpreter** — the isolation mechanism (UM-NATOS-001 §4.2)
Display framebuffer	DRAM	Large; 240×320×2 B = 150 KB for a full 16-bit buffer, which does not fit alongside everything else and will force partial-buffer rendering

The framebuffer figure is worth noting now: a full-screen 16-bit buffer is roughly 150 KB against ~176 KB of usable DRAM. Any display work must use partial buffers, and this constrains the graphics design more than any other single number in this report.

5. RESOLVED — bootloader IRAM overlap

Status: investigated and closed 2026-08-14. No action required. Original assessment ("high severity, blocks M1") was incorrect.

The measured boot trace (UM-NATOS-002 §4) shows the second-stage bootloader loading its own segments to:

```
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
```

The third segment lands at **`0x40080400`** — which is **inside the region nat-os declares as its IRAM** (`0x40080000` , length `0x20000`).

The kernel's `.text` currently occupies `0x40080000` – `0x400802E0` , i.e. 736 bytes, ending **288 bytes short of `0x40080400`** .

5.1 Why the overlap is harmless — the bootloader is split deliberately

The ESP-IDF bootloader links itself into **two** IRAM segments:

```
iram_loader_seg : org = 0x40078000, len = 0x8000   /* 32KB, APP CPU cache */
iram_seg        : org = 0x40080400, len = 0xfc00
```

Section placement, from the same script:

Segment	Receives
`iram_loader_seg` @ `0x40078000`	`.iram1.*` , the loader support code — **the code that copies the application**
`iram_seg` @ `0x40080400`	`.entry.text` , `.init` — early startup only

The bootloader's own commentary states the intent directly:

> "The main purpose is to make sure the bootloader can load into main memory without overwriting itself."
>
> "IRAM POOL1, used for APP CPU cache. Bootloader runs from here during the final stage of loading the app because APP CPU is still held in reset."

By the time application segments are copied, execution has moved to `iram_loader_seg` . The region at `0x40080400` holds initialisation code that has already completed and is *expected* to be overwritten. This is why ESP-IDF's own applications place `iram0_0_seg` at `0x40080000` and routinely occupy far more than 1 KB of it.

5.2 Empirical confirmation

Documentation was not treated as sufficient. A kernel was built with ~24 KB forced into `.text` so its IRAM segment spanned well past the disputed boundary, with sentinel words at both ends of the span.

Image size	26,048 bytes (vs 1,216 baseline)
IRAM span	`0x4008025C` … `0x40086258` — crosses `0x40080400` by ~24 KB
First sentinel	intact
Last sentinel	intact
Boot	normal; all M0 assertions still pass; heartbeat advancing

Result: the entire span was copied intact and the kernel ran normally. The overlap does not corrupt anything.

5.3 Corrected guidance

- The IRAM origin of `0x40080000` is **correct** and should not be changed.
- Moving it to `0x40088000` — the previously recommended "cheap mitigation" — would have discarded 32 KB of IRAM to solve a problem that does not exist.
- **M1 is not blocked by this.**

5.4 Note on the original error

The risk was raised from the boot trace alone: an address in the log fell inside a region the linker script claimed, and the conclusion was drawn from geometry without consulting the bootloader's design. The lesson for later milestones is that an apparent conflict in an address map is a prompt to read the other party's link script, not to move our own.

6. Unverified assumptions

Assumption	Status
DRAM above `0x3FFE0000` is ROM-reserved	Believed; not independently confirmed
IRAM window `0x40080000` + `0x20000` is fully available	**Contradicted by §5**
SRAM region boundaries in §2	Transcribed, not verified against silicon
Cache consumption of SRAM0 when XIP is enabled	Not measured — relevant from the first flash-resident build

7. References

- UM-NATOS-002 §4 — measured boot trace containing the overlapping load address
- UM-NATOS-006 — hardware confirmation of the addresses in §3.1
- ESP32 Technical Reference Manual — system address mapping